VIT-AP UNIVERSITY, ANDHRA PRADESH

Lab Sheet 11: Node JS basics

Branch/ Class: B.Tech/M.Tech **Date:**
Faculty Name: Prof. S.Gopikrishnan **School:** SCOPE
Student name: **Reg. no.:**

Exercise 1: Develop a Node.js application that provides a RESTful API for managing resources such as users or products. The application should support multiple HTTP methods and route handling. This exercise introduces structured backend development using Express.

Functional Requirements

- Create a Node.js application using the Node.js runtime environment
- Build the server using the Express.js framework
- Initialize the application using express() instance creation
- Define multiple routes using Express routing mechanism (app.get, app.post, app.put, app.delete)
- Handle different HTTP methods using REST API design principles
- Accept and process request data using request object in Express
- Send structured responses using response object (res.json, res.send)
- Parse incoming JSON data using express.json() middleware
- Implement route parameters using dynamic routing in Express
- Test API endpoints using HTTP client tools (e.g., browser or API tester)
- Maintain modular structure using separation of routes and logic

VIT-AP UNIVERSITY, ANDHRA PRADESH

Lab Sheet 11: Node JS basics

Branch/ Class: B.Tech/M.Tech **Date:**
Faculty Name: Prof. S.Gopikrishnan **School:** SCOPE
Student name: **Reg. no.:**

Lab_12 > ex-1 > app.js > ...

```javascript
const express = require("express");
const app = express();

// Global middleware
app.use((req, res, next) => {
  console.log(`Method: ${req.method}, URL: ${req.url}, Time: ${new Date()}`);
  next();
});

// Route-level middleware
const checkAuth = (req, res, next) => {
  console.log("Auth middleware executed");
  next();
};

// Route
app.get("/home", checkAuth, (req, res) => {
  res.send("Welcome to Home Page");
});

app.listen(3000, () => {
  console.log("Server running on http://localhost:3000");
});
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\SYS\Desktop\Web_tecnologies_lab_work\Lab_12\ex-1> node app.js
Server running on http://localhost:3000
Method: GET, URL: /, Time: Tue Apr 07 2026 22:09:27 GMT+0530 (India Standard Time)
Method: GET, URL: /, Time: Tue Apr 07 2026 22:09:32 GMT+0530 (India Standard Time)
```

Exercise 2: Develop a Node.js application that demonstrates middleware functionality. The application should log request details and control the flow of execution across multiple middleware layers.

Functional Requirements

VIT-AP UNIVERSITY, ANDHRA PRADESH

Lab Sheet 11: Node JS basics

Branch/ Class: B.Tech/M.Tech **Date:**
Faculty Name: Prof. S.Gopikrishnan **School:** SCOPE
Student name: **Reg. no.:**

- Create a Node.js server using the Express.js framework
- Implement middleware functions using app.use() in Express
- Log request details (method, URL, timestamp) using custom middleware function
- Control request flow using the next() function in middleware
- Create multiple middleware layers using middleware chaining in Express
- Apply middleware globally using application-level middleware
- Apply middleware to specific routes using route-level middleware
- Handle request preprocessing using middleware-based request handling
- Display middleware execution results using console logging
- Ensure proper execution order of middleware using Express middleware flow

VIT-AP UNIVERSITY, ANDHRA PRADESH

Lab Sheet 11: Node JS basics

Branch/ Class: B.Tech/M.Tech **Date:**
Faculty Name: Prof. S.Gopikrishnan **School:** SCOPE
Student name: **Reg. no.:**

Lab_12 > ex-2 > middleware.js > ...

```javascript
const express = require("express");
const app = express();

// Global middleware
app.use((req, res, next) => {
  console.log(`Method: ${req.method}, URL: ${req.url}, Time: ${new Date()}`);
  next();
});

// Route-level middleware
const checkAuth = (req, res, next) => {
  console.log("Auth middleware executed");
  next();
};

// Route
app.get("/home", checkAuth, (req, res) => {
  res.send("Welcome to Home Page");
});

app.listen(3000, () => {
  console.log("Server running on http://localhost:3000");
});
```

Exercise 3: Develop a Node.js application that connects to a database and performs CRUD operations. The application should store and retrieve data dynamically. This exercise introduces backend persistence using MongoDB.

Functional Requirements

- Create a Node.js application using the Node.js runtime environment
- Connect to the database using MongoDB with a Node.js driver or ODM
- Use an ODM like Mongoose for schema modeling
- Define a schema using Mongoose schema definition
- Create a model using Mongoose model creation

VIT-AP UNIVERSITY, ANDHRA PRADESH

Lab Sheet 11: Node JS basics

Branch/ Class: B.Tech/M.Tech **Date:**
Faculty Name: Prof. S.Gopikrishnan **School:** SCOPE
Student name: **Reg. no.:**

- Insert data into the database using create() or save() methods
- Retrieve data using find() method
- Update records using updateOne() or findByIdAndUpdate()
- Delete records using deleteOne() or findByIdAndDelete()
- Handle asynchronous database operations using async/await in Node.js
- Manage database connection using connection handling in Mongoose
- Return database responses through API using Express response handling

Lab_12 > ex-3 > app.js > ...

```javascript
const express = require("express");
const User = require("./model");

const app = express();
app.use(express.json());

// CREATE
app.post("/users", async (req, res) => {
  const user = await User.create(req.body);
  res.json(user);
});

// READ
app.get("/users", async (req, res) => {
  const users = await User.find();
  res.json(users);
});

// UPDATE
app.put("/users/:id", async (req, res) => {
  const user = await User.findByIdAndUpdate(req.params.id, req.body, { new: true });
  res.json(user);
});

// DELETE
app.delete("/users/:id", async (req, res) => {
  await User.findByIdAndDelete(req.params.id);
  res.send("Deleted");
});

app.listen(3000, () => {
  console.log("Server running on http://localhost:3000");
});
```

VIT-AP UNIVERSITY, ANDHRA PRADESH

Lab Sheet 11: Node JS basics

Branch/ Class: B.Tech/M.Tech **Date:**
Faculty Name: Prof. S.Gopikrishnan **School:** SCOPE
Student name: **Reg. no.:**